

1. MapScreen.tsx — Versión móvil (Android / iOS)

Este archivo implementa la pantalla del mapa para dispositivos móviles. Cuando la app corre en Android o iOS, Expo carga este archivo. En el navegador web se usa MapScreen.web.tsx en su lugar.

1.1 Importaciones

```
import * as Location from 'expo-location';
import MapView, { Region, UriTile } from 'react-native-maps';
```

expo-location proporciona acceso al GPS del dispositivo. react-native-maps es la librería que renderiza el mapa nativo. UriTile permite superponer imágenes de mapa externas (tiles) sobre el componente MapView.

1.2 Coordenadas por defecto (Sevilla)

```
const SEVILLA: Region = {
  latitude: 37.3886,
  longitude: -5.9823,
  latitudeDelta: 0.08,
  longitudeDelta: 0.08,
};
```

Define la región inicial del mapa: Sevilla. latitudeDelta y longitudeDelta controlan el nivel de zoom; un valor más grande aleja la vista. 0.08 equivale a ver un radio de unos 8 km alrededor del centro. Si el usuario deniega el permiso de ubicación, el mapa permanece aquí.

1.3 Estado del componente

```
const mapRef = useRef<MapView>(null);
const [userLocation, setUserLocation] = useState<
  { latitude: number; longitude: number } | null
>(null);
```

mapRef es una referencia al componente MapView que permite controlarlo desde el código (por ejemplo, mover la cámara con animación). userLocation guarda las coordenadas del usuario una vez obtenidas; empieza como null porque no se conocen hasta que el GPS responde.

1.4 Solicitud de permiso y obtención de ubicación

```
useEffect(() => { pedirUbicacion(); }, []);

async function pedirUbicacion() {
  const { status } =
    await Location.requestForegroundPermissionsAsync();
  if (status !== 'granted') return;

  const location = await Location.getCurrentPositionAsync({
    accuracy: Location.Accuracy.Balanced,
  });
  const coords = {
    latitude: location.coords.latitude,
    longitude: location.coords.longitude,
```

```
};
setUserLocation(coords);
mapRef.current?.animateToRegion(
  { ...coords, latitudeDelta: 0.01, longitudeDelta: 0.01 }, 800
);
}
```

useEffect con array vacío [] ejecuta pedirUbicación una sola vez al montar el componente. La función primero pide permiso al sistema operativo mediante requestForegroundPermissionsAsync; si el usuario lo deniega, la función termina y el mapa permanece en Sevilla. Si se concede, obtiene la posición actual con Accuracy.Balanced (equilibrio entre precisión y consumo de batería) y anima el mapa hasta las coordenadas del usuario en 800 ms con un zoom más cercano (latitudeDelta 0.01 $\approx$ 1 km de radio).

1.5 Botón de recentrar

```
function handleRecentrar() {
  if (!userLocation) return;
  mapRef.current?.animateToRegion(
    { ...userLocation, latitudeDelta: 0.01, longitudeDelta: 0.01 },
    600
  );
}
```

Al pulsar el botón [O], el mapa vuelve a la posición del usuario con una animación de 600 ms. Si la ubicación aún no se ha obtenido (null), no hace nada. Esto evita errores si el usuario pulsa el botón antes de que el GPS haya respondido.

1.6 Tiles de CARTO en lugar de OpenStreetMap directo

```
<UrlTile
  urlTemplate="https://a.basemaps.cartocdn.com/rastertiles/voyager/{z}/{x}/{y}.png"
  maximumZ={19}
  flipY={false}
  zIndex={-1}
/>
```

UrlTile superpone imágenes de mapa en formato XYZ tiles sobre el componente nativo. Se usan tiles de CARTO en lugar de los servidores oficiales de OpenStreetMap porque react-native-maps no permite añadir cabeceras HTTP personalizadas (como User-Agent), y OpenStreetMap bloquea con error 403 las peticiones sin User-Agent válido. CARTO sirve los mismos datos OSM sin esa restricción y de forma gratuita. flipY=false es el sistema de coordenadas estándar (TMS). zIndex=-1 coloca los tiles por debajo de los marcadores.

2. MapScreen.web.tsx — Versión web (navegador)

Cuando la app se abre en un navegador, Expo carga este archivo en lugar de MapScreen.tsx. La razón es que react-native-maps no funciona en web, por lo que se utiliza un <iframe> con el visor oficial de OpenStreetMap.

2.1 Estado y coordenadas

```
const SEVILLA_LAT = 37.3886;  
const SEVILLA_LON = -5.9823;  
const DELTA = 0.04;  
  
const [coords, setCoords] = useState(  
  { lat: SEVILLA_LAT, lon: SEVILLA_LON }  
);
```

Al igual que en la versión móvil, el estado inicial es Sevilla. DELTA define el tamaño del rectángulo visible en el mapa; 0.04 grados equivalen a unos 4 km de radio. Cuando se obtiene la ubicación del usuario, coords se actualiza y el iframe se recarga con las nuevas coordenadas.

2.2 Geolocalización en el navegador

```
function obtenerUbicacion() {  
  if (!('geolocation' in navigator)) return;  
  navigator.geolocation.getCurrentPosition((posicion) => {  
    setCoords({  
      lat: posicion.coords.latitude,  
      lon: posicion.coords.longitude,  
    });  
  });  
}
```

En el navegador no existe expo-location, así que se usa la API estándar de geolocalización del navegador (navigator.geolocation). La comprobación 'geolocation' in navigator garantiza compatibilidad con navegadores antiguos o entornos sin soporte. Cuando el navegador devuelve la posición, se actualiza el estado coords y React vuelve a renderizar el componente con el nuevo mapa.

2.3 Construcción de la URL del iframe

```
const bbox = `${coords.lon - DELTA},${coords.lat - DELTA}`,  
  + `${coords.lon + DELTA},${coords.lat + DELTA}`;  
  
const urlMapa = `https://www.openstreetmap.org/export/embed.html`  
  + `?bbox=${bbox}&layer=mapnik&marker=${coords.lat},${coords.lon}`;
```

bbox (bounding box) define el rectángulo visible del mapa como lon_izquierda, lat_abajo, lon_derecha, lat_arriba. layer=mapnik especifica el estilo de mapa estándar de OSM. marker añade un pin rojo en las coordenadas centrales. Cada vez que coords cambia, React recalcula urlMapa y el iframe se actualiza automáticamente al nuevo src.

2.4 El componente iframe

```
{/* @ts-ignore */}  
<iframe  
  src={urlMapa}  
  style={{ width: '100%', height: '100%', border: 'none' }}>
```

```
title="Mapa"  
/>
```

iframe es una etiqueta HTML nativa que incrusta otra página web. React Native no conoce este elemento (solo existe en web), por eso TypeScript necesita el comentario `@ts-ignore` para no dar error de tipos. El iframe funciona porque en la versión web Expo compila el código como HTML/CSS/JS estándar del navegador. El uso del embed de OSM está explícitamente permitido por la política de OpenStreetMap para proyectos pequeños.

3. ¿Por qué dos archivos para la misma pantalla?

Expo utiliza una convención de nombres para cargar archivos distintos según la plataforma: si existe `MapScreen.web.tsx`, se carga en el navegador; si existe `MapScreen.tsx`, se carga en Android e iOS. Esto permite usar la librería más adecuada en cada entorno sin condicionales en el código.

Resumen de diferencias

Movil (MapScreen.tsx)	Web (MapScreen.web.tsx)
-----	-----
react-native-maps	<iframe> de OSM
expo-location (GPS nativo)	navigator.geolocation
Tiles de CARTO via UrlTile	URL embed de OSM
Animacion nativa del mapa	Recarga del iframe

La lógica de negocio es idéntica en ambos archivos: pedir ubicación al arrancar, mostrar el mapa centrado en el usuario y ofrecer un botón para recentrar. Solo cambia la tecnología usada para renderizar el mapa, adaptada a las capacidades de cada plataforma.